KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

HOSTEL ROOM ALLOCATION & STUDENT STAY MANAGEMENT SYSTEM

Submitted by

Mohammed Ashik I

25BCS234

Under the Guidance of

Mr.Dharaneesh

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **2**

DECLARATION

I hereby declare that the project entitled **HOSTEL ROOM ALLOCATION & STUDENT STAY MANAGEMENT SYSTEM** submitted in partial fulfillment of the requirements for the course **Java programming** is a **record of my original work** carried out under the guidance of **Mr,Dharaneesh** Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

Mohammed Ashik I

CERTIFICATE

This is to certify that the project entitled **HOSTEL ROOM ALLOCATION & STUDENT STAY MANAGEMENT SYSTEM** is a **bona fide work** carried out by **Mohammed Ashik I ,1-C/25-26, BSC.Computer science, , KG College of Arts & Science,**, in partial fulfillment of the requirements for the course **Java programming**

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: Coimbatore

Date:10/4/2026

Submitted for the Viva-Voce Examination held on **10/4/2026**

Guide:
(Technical Trainer, KGCAS)

Mr.Dharaneesh

Head of the Department:
(Technical Head, KGCAS)

Dr.Boopalan

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

ABSTRACT

The Hostel Room Allocation & Student Stay Management System is a C-based application developed to manage hostel student records and room allocation efficiently. In many hostels, student data and room assignments are maintained manually, which can lead to errors, duplication, and difficulty in accessing information.

This system provides a simple and effective solution to automate the process of storing student details, assigning rooms, and managing their stay duration. The application uses file handling techniques to store and retrieve data, ensuring persistence of records.

The main objective of this system is to reduce manual effort, improve accuracy, and provide a user-friendly interface for hostel administrators. The project demonstrates the use of structures, file handling, and modular programming in C.

1. INTRODUCTION

- Hostel management plays a vital role in educational institutions. Managing student records and room allocation manually can be inefficient and time-consuming. This project introduces a computerized system that simplifies hostel management operations.
- The system allows users to:
- Add student details
- Allocate rooms
- View records
- Search for specific students

  By automating these tasks, the system improves efficiency and reduces human errors.

• 1.1 OBJECTIVE

- To develop a system for managing hostel student records
- To automate room allocation
- To store and retrieve student data using files
- To reduce manual errors and improve efficiency
- To implement C programming concepts practically

• 1.2 OVERVIEW

- This is a menu-driven application where the user can:
- Register students
- Assign room numbers
- Store duration of stay
- Search student records

  The system stores data in a file (`hostel.txt`) and retrieves it when required.

• 1.3 FEATURES

- Student Registration
- Room Allocation
- View All Records
- Search Student by ID
- File Storage System
- Simple User Interface

2. PROGRAM ANALYSIS

2.1 Problem Definition

Manual hostel management systems face several problems:

- Difficulty in maintaining records
- Errors in room allocation
- Time-consuming search process
- Data loss risk

This system solves these issues through automation.

2.2 Project Overview

The system uses:

- Structures to store student data
- File handling for storage
- Functions for modular design

It provides a simple interface for managing hostel operations.

2.3 Scope and Limitations

Scope:

- Small hostels
- Educational institutions
- Basic record management

Limitations:

- No database integration
- No GUI
- Limited to basic operations

3. SYSTEM DESIGN

3.1 MODULES

1. Student Registration Module

- Collects student details (Name, ID, Course)
- Stores data in file

2. Room Allocation Module

- Allocates rooms based on availability
- Avoids duplicate allocation

3. Room Management Module

- Tracks available and occupied rooms

4. Stay Management Module

- Maintains student stay details
- Handles check-in and check-out

5. File Handling Module

- Stores and retrieves data

4. METHODOLOGY

4.1 PROBLEM ANALYSIS

- Identified issues in manual systems
- Need for automation

4.2 SYSTEM DESIGN

- Divided system into modules
- Designed flow of allocation process

4.3 PROGRAM DEVELOPMENT

- Used:
    - Structures
    - Functions
    - File handling
    - Loops & conditions

4.4 TESTING AND VALIDATION

- Verified room allocation logic

- Checked data storage
- Tested multiple inputs

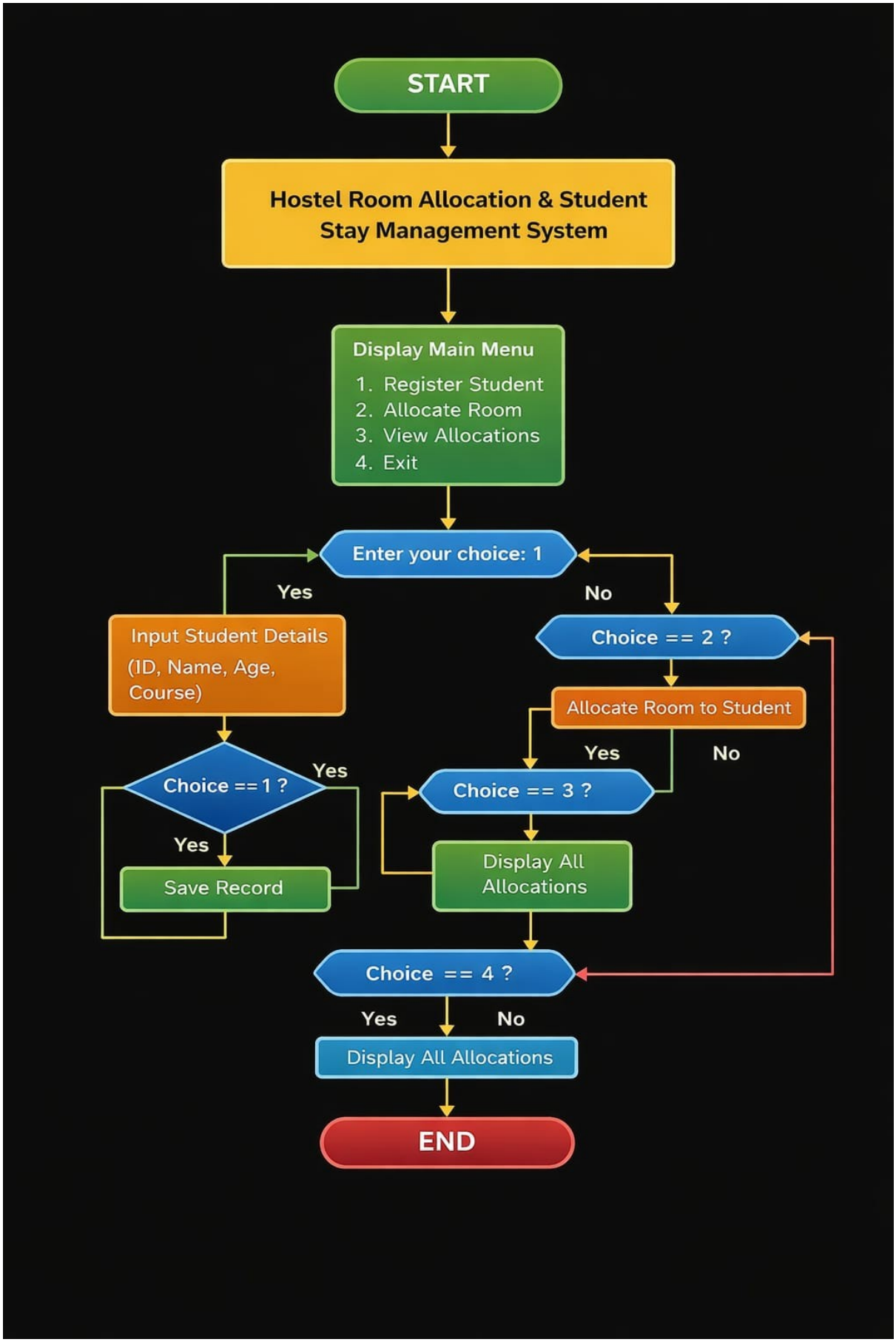
4.5 RESULT AND DOCUMENTATION

- System works successfully
- Data stored and retrieved correctly

4.6 ALGORITHM

1. Start
2. Display menu
3. Enter choice
4. If register → Enter student details
5. If allocate → Check room availability
6. Allocate room
7. Store data in file
8. Display result
9. Exit

4.7 FLOWCHART



5. SOURCE CODE

```
import java.io.*;
import java.util.*;

class Student {
    int id;
    String name;
    int age;
    int roomNo;
    int days;

    Student(int id, String name, int age, int roomNo, int days) {
        this.id = id;
        this.name = name;
        this.age = age;
        this.roomNo = roomNo;
        this.days = days;
    }
}

public class HostelManagement {

    static Scanner sc = new Scanner(System.in);
    static final String FILE_NAME = "hostel.txt";

    // Add Student
```

```
static void addStudent() {
    try (BufferedWriter bw = new BufferedWriter(new FileWriter(FILE_NAME, true))) {

        System.out.print("\nEnter Student ID: ");
        int id = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Name: ");
        String name = sc.nextLine();

        System.out.print("Enter Age: ");
        int age = sc.nextInt();

        System.out.print("Enter Room Number: ");
        int roomNo = sc.nextInt();

        System.out.print("Enter Number of Days: ");
        int days = sc.nextInt();

        bw.write(id + " " + name + " " + age + " " + roomNo + " " + days);
        bw.newLine();

        System.out.println("\nStudent Added Successfully!");

    } catch (IOException e) {
        System.out.println("Error writing to file!");
    }
}
```

```
// View Students
static void viewStudents() {
    File file = new File(FILE_NAME);

    if (!file.exists()) {
        System.out.println("\nNo records found!");
        return;
    }

    System.out.println("\n--- Student Records ---");

    try (Scanner fileScanner = new Scanner(file)) {

        while (fileScanner.hasNextLine()) {
            String line = fileScanner.nextLine();
            String[] data = line.split(" ");

            if (data.length >= 5) {
                System.out.println("\nID: " + data[0]);
                System.out.println("Name: " + data[1]);
                System.out.println("Age: " + data[2]);
                System.out.println("Room No: " + data[3]);
                System.out.println("Days: " + data[4]);
                System.out.println("----------------------");
            }
        }

    } catch (Exception e) {
        System.out.println("Error reading file!");
```

```
    }
}

// Search Student
static void searchStudent() {
    File file = new File(FILE_NAME);

    if (!file.exists()) {
        System.out.println("\nNo records found!");
        return;
    }

    System.out.print("\nEnter Student ID to search: ");
    int searchId = sc.nextInt();

    boolean found = false;

    try (Scanner fileScanner = new Scanner(file)) {

        while (fileScanner.hasNextLine()) {
            String line = fileScanner.nextLine();
            String[] data = line.split(" ");

            if (data.length >= 5 && Integer.parseInt(data[0]) == searchId) {
                System.out.println("\nRecord Found:");
                System.out.println("Name: " + data[1]);
                System.out.println("Age: " + data[2]);
                System.out.println("Room No: " + data[3]);
                System.out.println("Days: " + data[4]);
```

```
                found = true;
                break;
            }
        }

    } catch (Exception e) {
        System.out.println("Error reading file!");
    }

    if (!found) {
        System.out.println("\nRecord Not Found!");
    }
}

// Main Method
public static void main(String[] args) {

    while (true) {
        System.out.println("\n--- Hostel Management System ---");
        System.out.println("1. Add Student");
        System.out.println("2. View Students");
        System.out.println("3. Search Student");
        System.out.println("4. Exit");
        System.out.print("Enter your choice: ");

        int choice;

        try {
            choice = sc.nextInt();
```

```
            } catch (InputMismatchException e) {
                System.out.println("Invalid input!");
                sc.nextLine();
                continue;
            }

            switch (choice) {
                case 1: addStudent(); break;
                case 2: viewStudents(); break;
                case 3: searchStudent(); break;
                case 4: System.exit(0);
                default: System.out.println("Invalid choice!");
            }
        }
    }
}
```

6. INPUT AND OUTPUT SCREENS

Include screenshots of:

- Student registration
- Room allocation
- Room status
- Records display

7. DISCUSSION & CONCLUSION

7.1 DISCUSSION

The system simplifies hostel management by automating student registration and room allocation. It reduces workload and ensures accuracy.

7.2 CONCLUSION

The project successfully:

- **Automates room allocation**
- **Maintains student records**
- **Improves efficiency**

Future Enhancements

- **GUI interface**
- **Database integration**
- **Online booking system**

8. REFERENCES

Books

- *Programming with Java* – E. Balagurusamy
- *Java: The Complete Reference* – Herbert Schildt
- *Head First Java* – Kathy Sierra and Bert Bates
- *Core Java Volume I – Fundamentals* – Cay S. Horstmann

Websites

- Oracle – https://docs.oracle.com/javase
- GeeksforGeeks – https://www.geeksforgeeks.org
- W3Schools – https://www.w3schools.com
- Javatpoint – https://www.javatpoint.com